D1 - 1st release of the integrated scheduler

Project name
ANDREAS

Agreement Number

Duration
M1 - M6

Deliverable reference
D1

Date of delivery
M6

Responsible partner
POLIMI

Editor and authors
Danilo Ardagna (editor), Federica Filippini, Katarzyna Materka, Paweł Skrzypek

Reviewers
Fabrizio Magugliani

Distribution
Public

Executive Summary
This deliverable describes the first release of the GPU scheduler developed within the ANDREAS project and includes a high-level description of the ANDREAS solution architecture, the details of the resource management scheduling algorithm implemented and its preliminary performance evaluation.

This deliverable is the results of the work achieved within tasks T1 - *Maturing performance & energy ML models,* and *T2 - EUBra-BigSEA scheduler integration with the SLURM queue* during the first six months of the project.

Preliminary results have shown how the scheduling policies are effective providing a cost reduction from 12% to 97% with respect to first principle heuristics.

D1 - 1st release of the integrated scheduler 1

1. Introduction 3

2. ANDREAS high level architecture 3

3. Sequence flow 5

4. API and data structures 6

4.1 API and integration 6

4.1.1 Jobs Optimizer API 6

4.1.2 Integration of the components and external elements 9

4.2 Database structure 9

5. Jobs Optimizer algorithms description 12

6. Optimizer preliminary performance evaluation 16

6.1 Experimental setup 16

6.2 Experimental results 18

7. Conclusion 21

8. References 22

1. Introduction

This deliverable describes the first release of the GPU[1] scheduler developed within the ANDREAS project and includes a high-level description of the ANDREAS solution architecture, the details of the resource management scheduling algorithm implemented and its preliminary performance evaluation. This deliverable is the results of the work achieved within tasks T1 - *Maturing performance & energy ML models,* and *T2 - EUBra-BigSEA scheduler integration with the SLURM queue* during the first six months of the project.

This document is organized as follows. The next section introduces the ANDREAS project high level architecture providing a description and presentation of the main components included in the architecture and the key technical information and assumptions that characterise the current prototype release. Section 3 describes the sequence flow of the workflows and functional calls among the architectural components while Section 4 details the API[2] interfaces and data structures. Section 5 provides a description of the algorithms used by the Jobs Optimizer component which implements the intelligence to optimize the training job's placement while its preliminary performance evaluation is reported in Section 6. Conclusions, ongoing and future works are finally drawn in Section 7.

2. ANDREAS high level architecture

This section presents the initial high-level architecture of the solution created within the ANDREAS project. The architecture is designed to handle the complete training process of machine learning models, including collecting profile data for the given jobs and optimizing execution. Figure 1 depicts the overall architecture, components and relationships among them. The architecture design follows microservices architecture principles [6]. Components are run within Docker[3] containers [7]. The primary interface between ANDREAS components is based on the REST[4] approach [8]. The data structures for API are provided using JSON[5] schema [9]. Detailed description of API is provided in Section 5 of this document. Machine learning jobs are provided as Docker containers, which are built using the reference Nvidia ARM containers with TensorFlow [10] or PyTorch[11] frameworks installed within container. The machine learning jobs are envisioned to use PyTorch or TensorFlow as machine learning libraries. The machine learning jobs are started by the Jobs Manager component using SLURM [12] commands. All diagrams presented in this deliverable follow UML[6] standard for modelling ICT systems [15].

[1] Graphical Processing Unit.
[2] Application Programming Interface.
[3] https://www.docker.com/
[4] Representational state transfer
[5] JavaScript Object Notation
[6] Unified Modelling Language

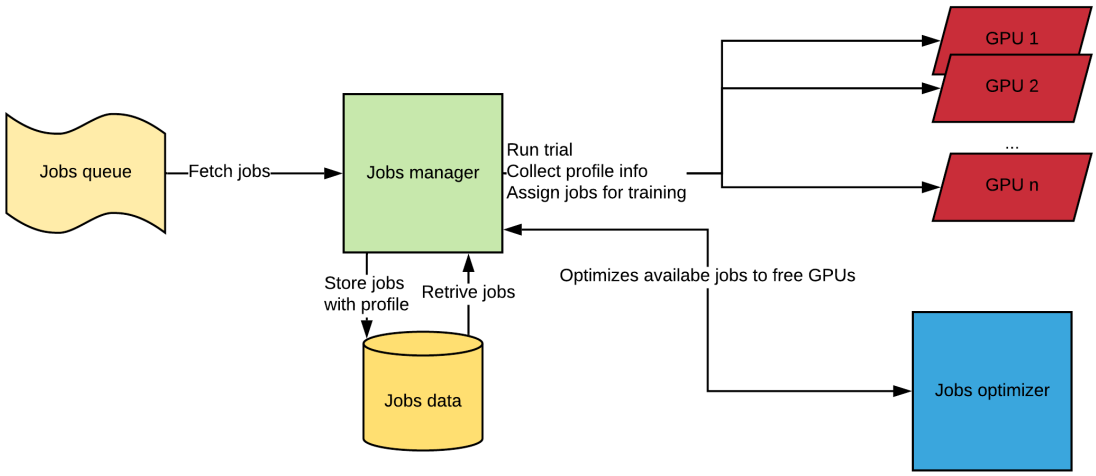

Figure 1. Initial ANDREAS high level architecture

The list and description of the components is presented in Table 1.

Id	Component	Description	Type
1.	Jobs queue	The queue where the job training requests are submitted.	Internal Andreas component
2.	Jobs manager	The component responsible for handling the workflow in ANDREAS.	Internal Andreas component
3.	Jobs optimizer	The component responsible for job's placement optimization.	Internal Andreas component
4.	Jobs data	Database to store jobs and profile information.	Internal Andreas component
5.	GPU	SLURM handled GPUs used to collect profile information and execute training.	External

Table 1. List and description of ANDREAS components

The relations among the components and their flow is presented in the next section.

3. Sequence flow

This section presents the complete sequence of actions in ANDREAS. The sequence is depicted on Figure 2 and described below. The sequence is presented in the form of UML Sequence diagram [15] which contains the main flow of the system.

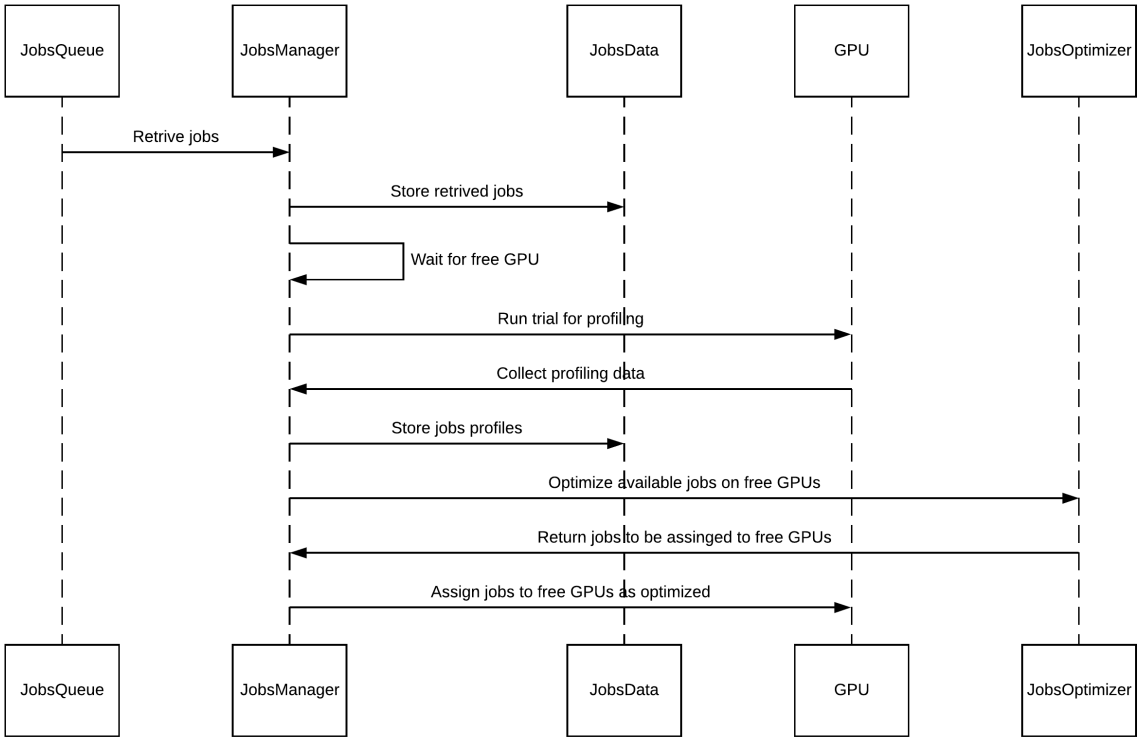

Figure 2. Sequence of actions in ANDREAS

Description of the flow:

1. A new machine learning training Job is submitted to the queue.
2. The Jobs Manager retrieves the Job.
3. The Jobs Manager stores Job-related information in JobsData database.
4. The Jobs Manager is waiting for the available GPU to start profiling operations.
5. The Jobs Manager is running Jobs to gather profiling information - according the profiling rules (currently per iteration execution time are gathered by running few training epochs)
6. The Jobs Manager stores profiling data in JobsData database.
7. The Jobs Manager invokes Jobs Optimizer to optimize Jobs execution.
8. The Optimizer optimizes Jobs execution providing the number, type of GPU to be used to run the job within its deadline, possibly minimizing energy consumption and job tardiness in case the deadline cannot be fulfilled. The Optimizer also identifies the node of the cluster where the Job (application) container has to be run.
9. The Jobs Manager runs Jobs according to the results of optimization.

The flow described above is implemented in the form of prototype release and is currently under test with benchmarking applications (see also Section 6).

4. API and data structures

This Section contains the description of API and integration methods (internal and external) in the ANDREAS and data structures used in the project. The data structures are described at logical and physical level and depicted in the respective diagrams.

4.1 API and integration

4.1.1 Jobs Optimizer API

The Jobs Optimizer described in Section 5 and implemented in C++ is accessible through a REST API implemented by relying on Flask[7]. The user interacts with the application through a POST request, whose details are presented hereunder.

POST /optimizer/v1	Call the optimizer, passing the relevant data in terms of jobs characteristics and node resources; receive the optimal assignment (node ID and number of GPUs) for all jobs, the expected cost and the expected rescheduling time.

[7] https://flask.palletsprojects.com/en/1.1.x/

Parameters:

Name	Description
body (required)	Set of jobs, node resources and parameters required for the optimization Model: {"jobs": {ID: {"SubmissionTime": "%a %d %b %Y, %H.%M.%S", "Deadline": "%a %d %b %Y, %H.%M.%S", "TardinessWeight": float, "IterationProgress": int/int, "ExecutionTimes": {string: {int: float, [s] int: float, [s] ...}, string: {int: float, [s] int: float, [s] ...}, ...} } }, "nodes": {ID: {"GPUtype": string, "nGPUs": int, "cost": float [$/h] } } "currentTime": "%a %d %b %Y, %H.%M.%S", "method": string, "verbose": int } Parameter content type: json Note (1): all fields except the method and the verbosity level are mandatory. Note (2): the ExecutionTimes section stores information collected through profiling, concerning the execution times of each job with the available configurations, identified by a given type and number of GPUs.

Responses:

Code	Output	Description
201	Response of the optimizer, specifying the estimated cost and rescheduling time and the optimal assignment for all jobs. Model: {"estimated_cost": float, "estimated_rescheduling_time": "%a %d %b %Y, %H.%M.%S", "jobs": {ID: {"expected_tardiness": float, [h] "nGPUs": int, "node": string } } } Output content type: json	POST success
414	Body provided by the user	KEY ERROR Mandatory field "jobs" missing in data
424	Body provided by the user	KEY ERROR Mandatory field "nodes" missing in data
434	Body provided by the user	KEY ERROR Mandatory field "currentTime" missing in data
444	Body provided by the user	PARAMETER ERROR Deadline or tardinessWeight parameter equal to 0 for at least one job
454	Body provided by the user	PARAMETER ERROR Missing data in the ExecutionTimes section
464	Body provided by the user	OPTIMIZER FAILURE

4.1.2 Integration of the components and external elements

The Jobs Manager component is responsible for the orchestration of the ANDREAS workflow as presented in Section 3. It utilizes the API of the Jobs Optimizer component described in the section 4.1.1 to communicate with the Jobs Optimizer. The communication with the database is done by using the JDBC protocol. The management of the jobs operations is done by invoking the respective SLURM commands to start the Docker container with the job on the selected GPU and by monitoring if the job is finished.

4.2 Database structure

The Jobs Database is built using the MariaDB [13] database engine. It contains the entities presented in the form of ERD diagram [14] in Figure 3 and described in Table 2. Figure 3 also presents the relations among entities. The physical scheme for the database is presented in Figure 4.

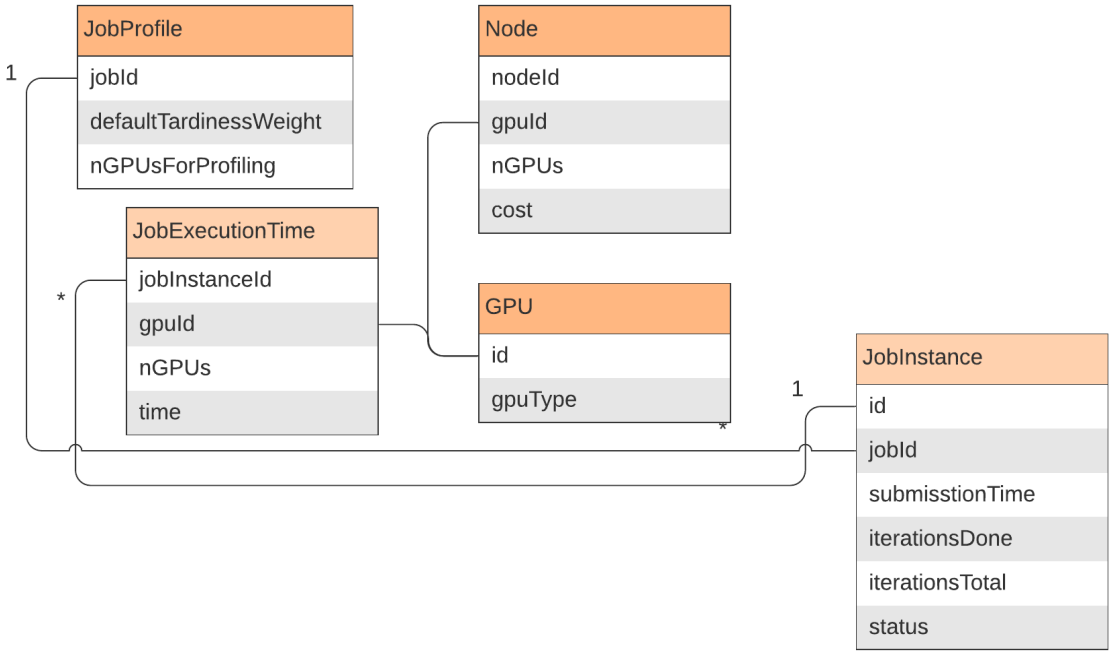

Figure 3. Entities and relationships as ERD diagrams.

Entity	Description	Fields

JobsProfile	Data related to job's profile.	JobId defaultTardinessWeight nGPUsForProfiling
JobsExecutionTime	Execution time of the jobs and information related to that.	JobInstanceId gpuId nGPUs time
Node	The definition of the Node.	nodeId gpuId nGPUs cost
GPU	The definition of the GPU.	Id gputype
JobsInstance	Data related to the job instance execution.	Id jobId submissionTime iterationsDone iterationsTotal status

Table 2. Entity description

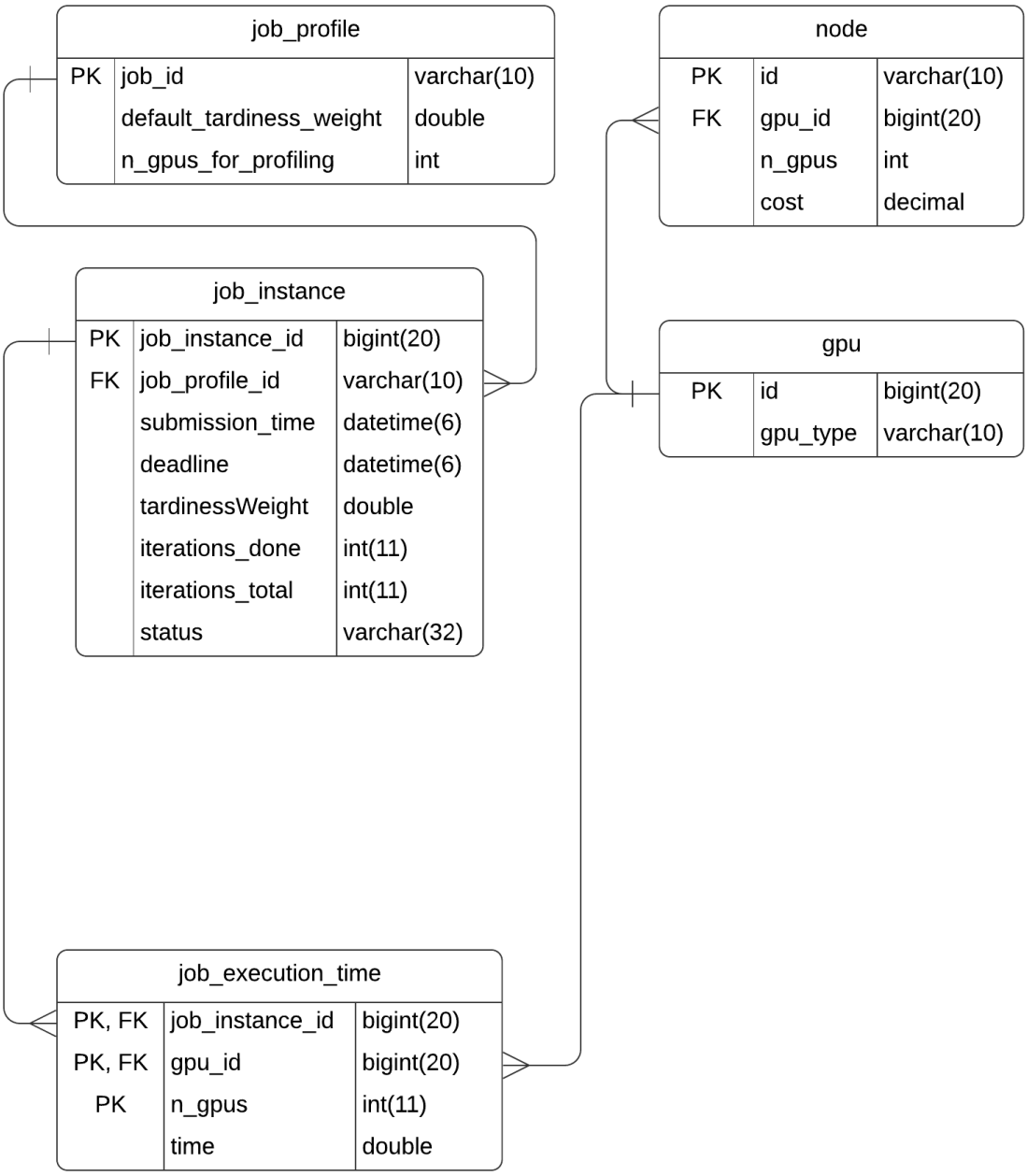

Figure 4. Physical database schema

The Jobs Database is used to collect and store profile information which are provided to the Jobs Optimizer component to perform optimization. Additional profile data, collected from job's trail runs, allows the Jobs Optimizer to increase the quality of the solution of the optimization.

5. Jobs Optimizer algorithms description

Formally, the reference system includes a set $N = \{n_1, n_2, \ldots, n_N\}$ ($N = |N|$) of job processing units (referred to as nodes); each node is characterized by a specific type and number of available GPUs, other than by a time-unit cost c_n. The number of GPUs available on each node is identified by a set $G_n = \{1, 2, \ldots, G_n\}$.
The set of submitted jobs (which includes the jobs in execution and in the waiting queue) at a given time instant is denoted by J. Each job is characterized by its release time, its due date d_j and by an estimated processing time t_{jng} that depends on the node n (and thus the type of GPU) and the number g of GPUs assigned to the job. Jobs are never rejected, but they can be preempted, and their execution can be postponed to a following time period if a job with higher priority is released and the available resources are insufficient to run both. t_{jng} is obtained by some pre-profiling runs which include few iterations of the target job (application) and stored in the **job_execution_time** table (see Figure 4). The priority of a job depends on the remaining processing time, the due date and the associated tardiness penalty. The tardiness of job $j \in J$ is denoted by τ_j whereas a weight ω_j is used to compute the tardiness penalty, $\omega_j\tau_j$.
At any time a new job j is submitted to the system or the execution of a running job is completed, all incomplete or not yet started jobs are placed in the same queue, sorted by release time, and their estimated execution time is updated. Then, the optimization problem is solved to determine the nodes to use. A selection of jobs from the queue is assigned to those nodes and, within each node, to a certain number of dedicated GPUs. The remaining jobs are kept in the queue until the following rescheduling point.
The job selection and resource allocation processes aim at executing all jobs in J before their due date and, at the same time, to maximize the utilization of the selected nodes avoiding idle resources, thus reducing the overall cost, which is given, as reported in Equation (1), by the sum of the execution costs of all jobs and of the tardiness cost of jobs whose due date is violated. Since this penalty term is always more significant than the execution cost per time unit, the system tends to avoid delays as much as possible, according to the available resources.

$$f_{\mathrm{OBJ}}: \quad \sum_{j \in \mathcal{J}} (\omega_j \tau_j + \rho \omega_j \hat{\tau}_j) + \sum_{\substack{j \in \mathcal{J} \\ n \in \mathcal{N}}} \alpha_j \pi_j \qquad (1)$$

To solve this joint Capacity Allocation and Job Scheduling problem, we have developed an heuristic algorithm based on a randomized greedy approach. The proposed method is based on the following assumptions.

- Jobs should be scheduled according to their pressure, which is an index that measures how close they are to the deadline when executed with the fastest configuration. Denoted by T_c the current scheduling point, for each job $j \in J$, the pressure Δ_j is computed as:

$$\Delta_j = T_c + \min_{\substack{n \in \mathcal{N} \\ g \in \mathcal{G}_n}} (t_{jng}) - d_j \qquad (2)$$

- The optimal configuration $(n,g) \in N \times G_n$ for each selected job is:

  - the cheapest configuration such that the job is executed before its due date, if such a configuration exists,
  - the fastest available configuration if, independently from the selected setup, it is not possible to execute the job before its due date.

  This assumption is justified by the fact that, in our scenario, the time unit deployment cost of a job on any available configuration is always lower than the penalty incurred if the job due date is violated. Therefore, it is reasonable to choose the cheapest configuration as long as the due date can be met, while, as soon as it is violated, it is more convenient to complete the job execution as fast as possible in order to reduce the corresponding penalty.
- Deployment costs increase linearly in the number of GPUs, as demonstrated by Cloud providers pricing models[8] [9] and according to GPU systems power consumptions [16].
- The speedup of job execution is sublinear in the number of GPUs, as observed in GPU-based application benchmarks [1].

The proposed method, reported in Algorithm (1), includes three main phases: in the *preprocessing* step, candidate jobs are sorted according to their pressure. Then, the *scheduling* step implements a randomized construction procedure, where the best available configuration is selected for all the jobs in the queue. Finally, the *postprocessing* step aims at reducing the amount of idle resources by modifying or reallocating the configurations selected on the different nodes.

Algorithm 1

Step#0 - PREPROCESSING

$T_c \leftarrow T_c + T$
$\mathcal{N}_O \leftarrow \emptyset$ ▷ $\mathcal{N}_O$: set of open nodes
for all $j \in J$ **do**
  if j was partially executed in previous steps **then**
    for all $n \in \mathcal{N},\ g \in \mathcal{G}_n$ **do**
      $t_{jng} \leftarrow t_{jng} \cdot (100 - \bar{c}_{p_j})/100$ ▷ $\bar{c}_{p_j}$: completion percentage
    end for
  end if
  $\Delta_j \leftarrow \min_{n,g}(t_{jng}) + T_c - d_j$
end for

Step#1 - SCHEDULING

$\mathcal{S}^* \leftarrow$ RANDOMIZED_CONSTRUCTION$(\mathcal{J}_s, \text{MAXIT})$

Step#2 - POSTPROCESSING

for all $\nu \in \mathcal{N}_O$ **do**
  if there exist idle GPUs on ν **then**
    Assign the idle GPUs to the job with the highest speed-up
  end if
end for

[8] Amazon Web Service pricing list: https://aws.amazon.com/ec2/pricing/on-demand/?nc1=h_ls, 2020
[9] Azure Cloud Services pricing list: https://azure.microsoft.com/en-us/pricing/details/cloud-services/, 2020

The overall costs of the proposed solutions are computed according to the formula reported in Equation (1), denoted as f_{OBJ}. The three phases of the algorithm are described in the following:

Step 0 - Preprocessing:

First, at lines 1 and 2 of Algorithm (1), the current time T_c is updated by adding the elapsed time T between the previous scheduling point and the current one, while the set N_O of *open* nodes is initialized with the empty set. A node $n \in N$ is denoted as *open* if at least one job j has been assigned to it. Then, the full set J of submitted jobs is considered. If a job has been partially executed in the previous time period, its processing time t_{jng} is updated for all possible values of $n \in N$ and $g \in G_n$.
In particular, being $\bar{c}_{pj}$ the completion percentage of job j, its processing time is modified, at line 6, by replacing its value with:

$$t_{jng} \cdot \frac{100 - \bar{c}_{p_j}}{100}$$

The job pressure is, then, computed at line 9 as defined in Equation (2).

Step 1 - Scheduling

The scheduling process is composed by a randomized greedy construction process, used to build a set S^* of candidate solutions with minimized local cost function f_{OBJ}. It is reported in Algorithm (2). At each iteration, a new candidate solution is built through a randomized greedy procedure and, if the corresponding cost is lower than the one of any solution currently stored in S^*, the set is updated accordingly, so that its total number of elements does not exceed a fixed value σ.

Algorithm 2 Randomized construction process

1: **function** RANDOMIZED_CONSTRUCTION($\mathcal{J}_s$, MAXIT)
2: iter = 0
3: $\mathcal{S}^* \leftarrow \emptyset$
4: **while** iter < MAXIT **do** ▷ MAXIT: maximum number of random iterations
5: $S \leftarrow$ empty schedule ▷ S: current schedule
6: $\mathcal{J}_s \leftarrow$ SORT_JOBS_LIST($\mathcal{J}, \Delta$) ▷ Δ : pressures of all jobs
7: **for all** $j \in \mathcal{J}_s$ **do** ▷ $\mathcal{J}_s$: sorted queue
8: $D_j^* = \{(n, g) \text{ s.t. } t_{jng} + T_c < d_j\}$
9: $(n^*, g^*) \leftarrow$ SELECT_BEST_CONFIGURATION(j, D_j^*)
10: assigned $\leftarrow$ ASSIGN($j, (n^*, g^*), \mathcal{N}_O$)
11: **if** not assigned **then**
12: ASSIGN_TO_SUBOPTIMAL($j, S, \mathcal{N}_O$)
13: **else**
14: $S \leftarrow S \cup \{[j, (n^*, g^*)]\}$
15: **end if**
16: $\mathcal{J}_s \leftarrow \mathcal{J}_s \setminus \{j\}$
17: **end for**
18: **if** $f_{\text{OBJ}}(S) < f_{\text{OBJ}}(S')$ for any $S' \in \mathcal{S}^*$ **then**
19: $\mathcal{S}^* \leftarrow \mathcal{S}^* \cup \{S\}$
20: $\mathcal{S}^* \leftarrow \mathcal{S}^* \setminus \{S'\}$ if $|\mathcal{S}^*| > \sigma$
21: **end if**
22: iter $\leftarrow$ iter + 1
23: **end while**
24: **return** $\mathcal{S}^*$
25: **end function**

As a first step, in line 6 of Algorithm (2), the set *J* of submitted jobs is sorted, producing as output a new set denoted by J_s. Note that job j precedes job k in J_s if and only if $\Delta_j > \Delta_k$, i.e., job j is more likely to violate its due date. As a first randomization step, some jobs in J_s can be swapped, with probability inversely proportional to their priority.
For all jobs j in the sorted list *Js*, the set of configurations (i.e., pairs made by a node and a certain number of GPUs) such that the job can be executed within its due date (determined by the condition $t_{jng} + T_c < d_j$) is defined, in line 8, as:

$$D_j^* = \{(n, g) \in \mathcal{N} \times \mathcal{G}_n \text{ s.t. } t_{jng} + T_c < d_j\}.$$

The set D^*_j is used to select the best configuration for job j, according to the following rules. Since one of the main contributions to the total cost of the schedule is the penalty due to deadline violations, we always try to select configurations that guarantee to complete the jobs without tardiness (or with the least possible delay, if there is no possibility to match the required due date). Therefore, we define the optimal configuration as:

$$(n^*, g^*) = \begin{cases} \text{argmin}_{D_J^*} (t_{jng} c_n) & \text{if } D_j^* \neq \emptyset \\ \text{argmin}_{n \in \mathcal{N}, g \in \mathcal{G}_n} (t_{jng}) & \text{otherwise} \end{cases}$$

In the first case, the set D^*_j is not empty, which entails that there exist configurations (n,g) such that the job can be executed before its due date. In this case (n*,g*) is identified by the cheapest configuration, i.e., by the element in D^*_j with minimum cost (identified by the product between the time-unit cost c_n and the expected execution time t_{jng}). In the second case, the job due date is violated with any available configuration, therefore we select the one that guarantees the minimum expected execution time, in order to reduce the tardiness as much as possible.
The algorithm explores the space of possible configuration by introducing some randomness in the the configuration selection process, where, instead of choosing the configuration

according to the rule defined above, it selects as candidate configuration for job j one of the (n, g) with lower cost, with probability inversely proportional to the cost itself. Having denoted by (n*,g*) the selected configuration for job j, the assignment proceeds as follows:

1. First of all, the algorithm tries to assign job j to the required node n*, provided that the number of available GPUs in n* is enough to assign g* of them to job j.
2. If this assignment is not feasible, it tries to assign job j to a suboptimal configuration available in an open node. Iteratively, it loops over the elements in D* until it finds a configuration that fits in any node n' $\in N_O$.

Step 2 - Postprocessing

The last step of the algorithm implements a postprocessing phase, aiming to reduce as much as possible the number of idle GPUs in open nodes. This goal is achieved by determining, among the jobs deployed on each node having idle GPUs, the one that would get the highest speed-up by receiving additional resources. The idle GPUs are then assigned to this job.

6. Optimizer preliminary performance evaluation

The heuristic method described in the previous section has been evaluated through an extensive experimental campaign, focusing both on the quality of the results and on the efficiency. In particular, this approach is compared against first-principle methods and the Hierarchical Method presented in [2].

6.1 Experimental setup

As representatives of long-running DL training jobs, we selected the training of different neural networks, both for image and speech recognition, namely Alexnet, Resnet, VGG, and DeepSpeech, implemented with different deep learning frameworks (i.e., PyTorch and TensorFlow). They are characterized by a significant heterogeneity in terms of resource usage, thus they can be considered representative samples of the variety of architectures used in practice for image classification and video processing.
We have considered several instances of the listed DL training applications, with different epochs numbers and batch sizes and therefore different characteristics in terms of expected execution times. These execution times have been estimated by relying on the Machine Learning models presented in [1]. Based on linear regression, these models allow users to learn the execution times of DL jobs, starting from a training set of profiling runs of the target applications themselves, with an average percentage error of about 10%.

Since the energy profiling activities are still on-going, the considered node set *N* (reported in Table (1)) is composed of 11 target elements. Six of them (N1 to N6) are based on Nvidia K80 and M60 and are analogous, in terms of number of available GPUs and time-unit costs, to Virtual Machines available on Microsoft Azure catalog. Three (N7 to N9) are based on

Politecnico di Milano in-house servers (Quadro P600 and GTX 1080Ti), while the last ones (N10 and N11) have been added to increase the complexity of the problems by enlarging the set of possible candidates (their characteristics have been obtained from N3 and N6, doubling the number of available GPUs and also doubling their hourly costs which is in line with the current cloud providers pricing models).

Table 1: Characteristics of the Target Nodes

Node ID	GPU type	# GPU	Cost [$/h]
N1	K80	1	0.56
N2	K80	2	1.13
N3	K80	4	2.25
N4	M60	1	0.62
N5	M60	2	1.24
N6	M60	4	2.48
N7	Quadro P600	2	0.11
N8	GTX 1080Ti	8	1.13
N9	Quadro P600	8	0.44
N10	K80	8	4.48
N11	M60	8	4.96

To verify the effectiveness and generality of our heuristic methods, several random problem instances have been generated using the parameters described in the following.

We varied the number N of available nodes in the cluster from 10 to 100, while the number of submitted jobs in each instance is set to J = 10N. Jobs inter-arrival times have been generated according to different rules:

- In the first set of simulations, inter-arrivals have been generated, as in other literature proposals [3], according to an exponential distribution, with average equal to 75,000s, divided by the number of nodes N in the system. The division by the number of nodes guarantees that the workload each node is subject to remains constant as the system size increases (to avoid saturation conditions).
- In other sets of simulations, we have generated inter-arrival times starting from the procedure described in [4]. Arrivals are always sampled from a Poisson distribution, considering three possible rates: given a base rate λ, defined as the reciprocal of the minimum expected completion time, a *high* rate has been generated as $\varepsilon k_{max}\lambda$, while a *low* rate as $\varepsilon k_{max}\lambda/4$. We have defined k_{max} as the number of nodes in the system multiplied by the maximum number of GPUs that can be assigned to each job. The parameter ε has been tuned to adapt the peak load of the system to real-life scenarios, with the same order of magnitude of the peak loads reported in [5]. Finally, a *mixed* inter-arrival rate has been obtained by alternating *high* and *low* distributions approximately every 10 submissions.

The distributions of jobs arrivals, in a sample system with N = 1000 and J = 10000, are reported in Figure 5.

For each value of the cluster size and for each arrival rate, we have generated three problem instances, changing the seed of random distributions. The remaining parameters are set as follows. The periodic scheduling time interval H is set to one hour. The due date d_j for each job is randomly generated according to a uniform distribution in the range $[\min(t_{jng}), 3\min(t_{jng})]$.

The tardiness weights are randomly generated as $\omega_j = 2o_j$, where o_j is extracted, with uniform probability, from the interval [0.0015, 0.0075]. Given the expected execution times of jobs and the time-unit costs of VMs, those values ensure that penalties for deadlines violations are always more expensive than execution costs. For what concerns the worst-case tardiness, the parameter ρ that, multiplied by ω_j in the objective function of the model (see Equation (1)), penalizes the postponement of jobs, is set to 100. Whenever the energy profiling activity of task T1 will be completed, the o_j parameter (currently setup according to cloud VM costs) will be fine-tuned according to the energy costs.

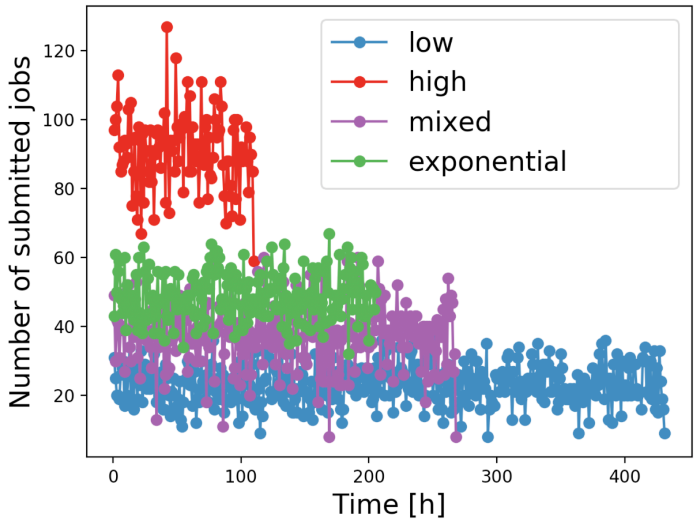

Figure 5 - Sample workloads considered in the simulation campaign

The results of the proposed approach have been compared against those obtained with first principle methods, such as First-in-First-out (FIFO), Earliest Deadline First (EDF), Priority Scheduling (PS), and with the Hierarchical Method described in [2]. All tests involving the Hierarchical Method have been performed in Python, by relying on Gurobi Optimizer 9.0. The relative mixed integer programming gap (the difference between the current upper and lower bounds of the MILP solver) has been set to 5%. This value guarantees, as in [17], a good trade-off between the quality of the solution built by the optimizer and its performance in terms of execution time.

6.2 Experimental results

The results of the analyzed approaches are compared in Figure 6 and 7. In particular, Figure 6 shows the average total costs obtained with the different methods in all the considered scenarios (identified by a different inter-arrival distribution as described in the previous section). The results of different experiments with the same number of nodes have been averaged and normalized with respect to the best result obtained by first-principle-methods (that, in our cases, corresponds to the result obtained by EDF).

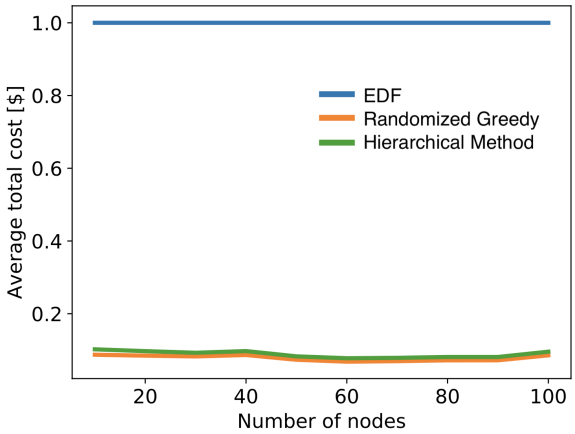

Figure 6.a - *exponential* inter-arrival times

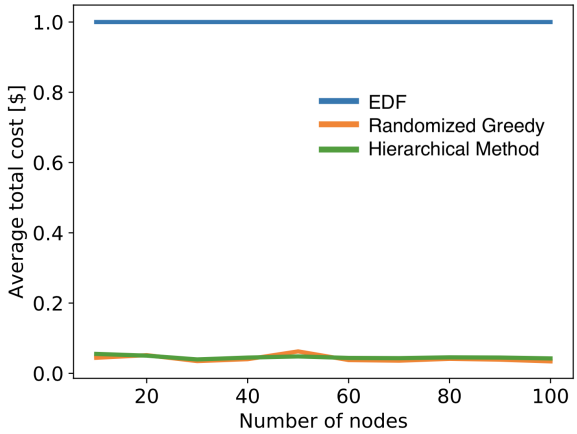

Figure 6.b - *high* inter-arrival rate

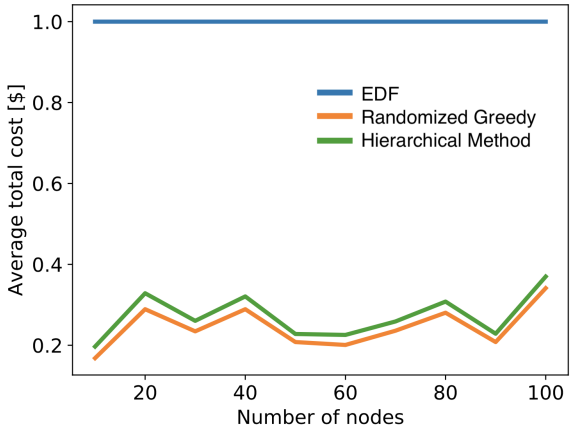

Figure 6.c - *mixed* inter-arrival rate

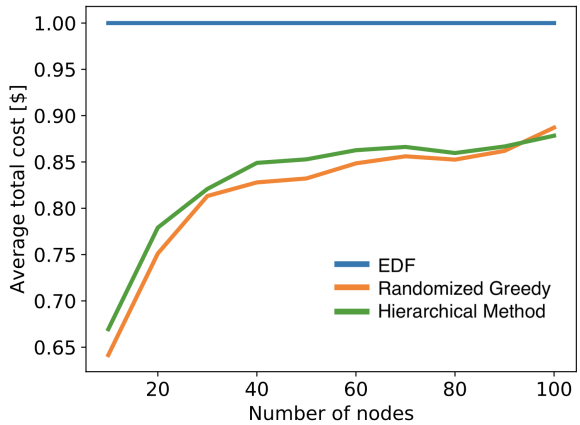

Figure 6.d - *low* inter-arrival rate

Figure 6 - Average total costs with the different methods

It is possible to notice that the average total cost obtained with our Randomized Greedy approach is always considerably lower than the one obtained by EDF. Moreover, this method guarantees better performance than the Hierarchical Method [2] in all considered scenarios. It is worth noticing that the highest difference between our results and the results of EDF is obtained in scenarios considering a higher inter-arrival rate. Indeed, in these cases, the system is subject to higher pressure and approaches that enable resource sharing guarantee better performance.

Figure 7 reports the average percentage gain obtained by all the methods in the different scenarios.

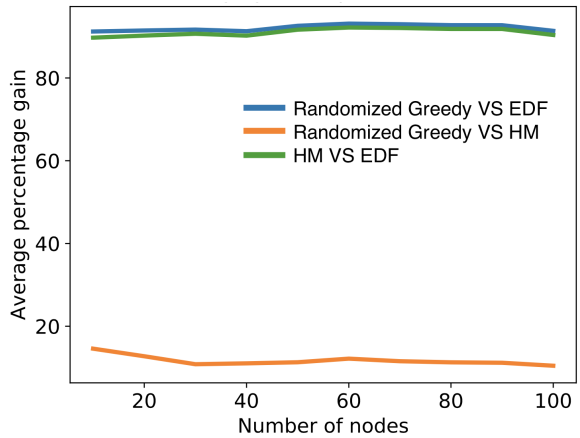

Figure 7.a - *exponential* inter-arrival times

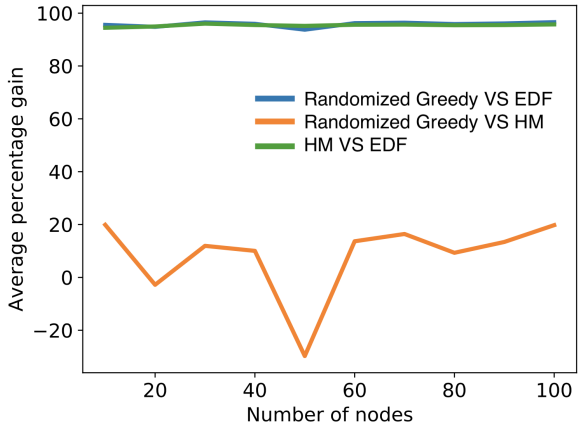

Figure 7.b - *high* inter-arrival rate

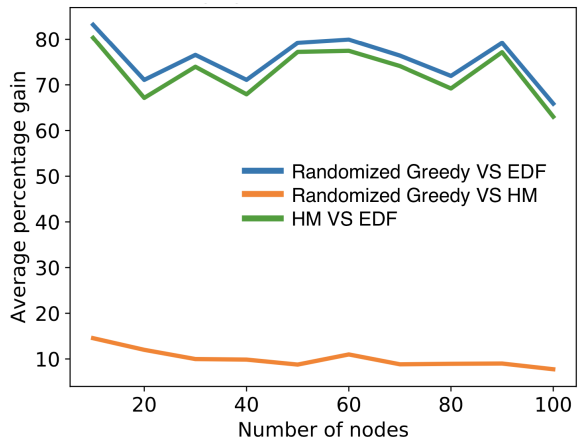

Figure 7.c - *mixed* inter-arrival rate

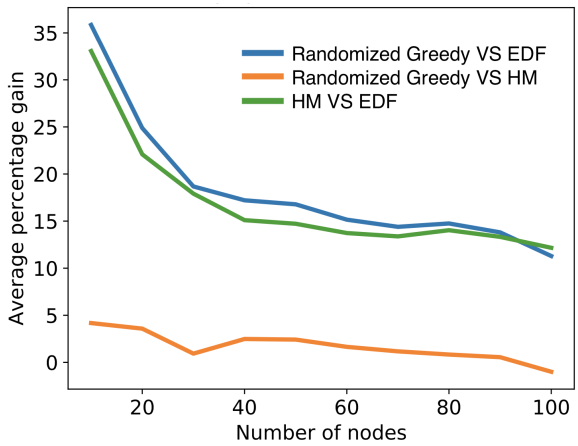

Figure 7.d - *low* inter-arrival rate

Figure 7 - Average percentage gain of the different methods

Finally, we have reported in Figure 8 the average execution time of each call to the Randomized Greedy algorithm. It is worth noticing that in all scenarios, except for the one with high inter-arrival rates, our method solves an instance of the problem involving up to 100 nodes and 1000 jobs in less than a second, on average. The slightly higher time required in the case of high inter-arrival rates is explained by the fact that the system is subject to a considerably higher pressure and therefore it is harder to determine a good quality solution.

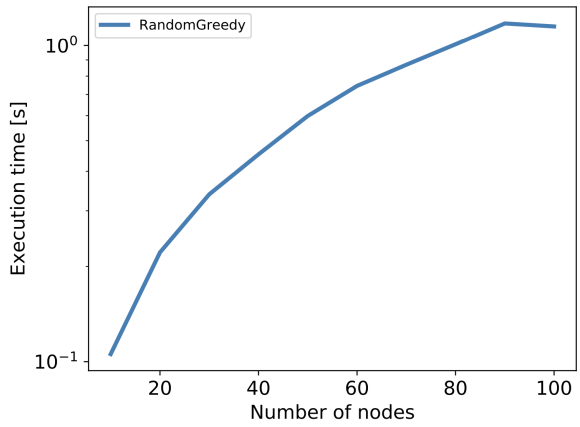

Figure 8.a - *exponential* inter-arrival times

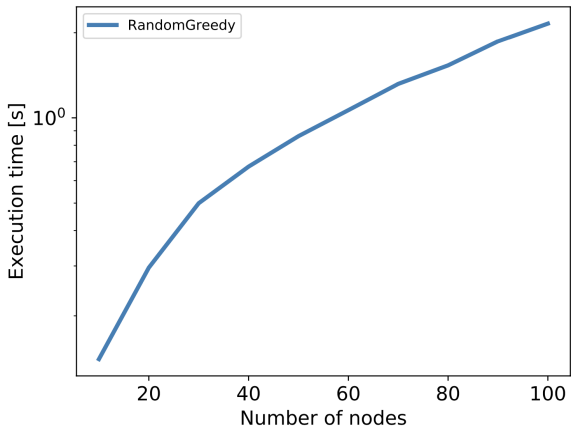

Figure 8.b - *high* inter-arrival rate

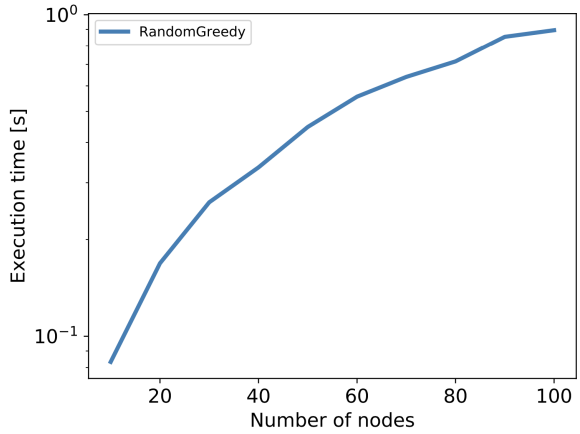

Figure 8.c - *mixed* inter-arrival rate

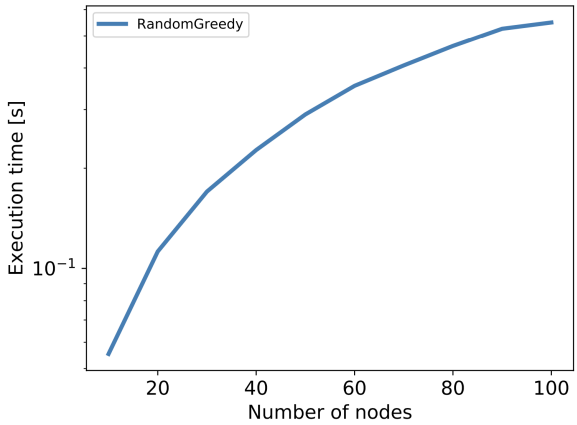

Figure 8.d - *low* inter-arrival rate

Figure 8 - Average execution time of each optimizer call

7. Conclusion

This document provides a snapshot of the current status of the ANDREAS project development activities. Preliminary results have shown how the scheduling policies are effective providing a cost reduction from 12% to 97% with respect to first principle heuristics and improving the research results we recently achieved and published in [2] by 8% on average.

Building upon these results, the project will focus on the integration with the SLURM queue and the ARMIDA eco-system (ARMIDA is a 8-node ARM cluster) and on the energy profiling of benchmark applications.

8. References

[1] Gianniti, E., Zhang, L., Ardagna, D., 2019, *Performance Prediction of GPU-based Deep Learning Applications*, In Proceedings of the 9th International Conference on Cloud Computing and Services Science (CLOSER 2019), pages 279-286.

[2] Filippini, F., Lattuada, M., Jahani, A., Ciavotta, M., Ardagna, D., Amaldi, E., *Hierarchical Scheduling in on-demand GPU-as-a-Service Systems*, 22nd International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC 2020). To Appear. 1-8. Timisoara, Romania.

[3] Amaral, M., Polo, J., Carrera, D., Seelam, S., Steinder, M., 2017, *Topology-aware GPU scheduling for learning workloads in cloud environments*, In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '17).

[4] Saxena, V., Jayaram, K. R., Basu, S., Sabharwal, Y., Verma, A., 2020, *Effective Elastic Scaling of Deep Learning Workloads*, ArXiv, abs/2006.13878.

[5] Peng, Y., Bao, Y., Chen, Y., Wu, C., Meng, C., Lin, W., 2019, *DL2: A Deep Learning-driven Scheduler for Deep Learning Clusters*, ArXiv, abs/1909.06040.

[6] Nadareishvili, I., Mitra, R., McLarty, M., & Amundsen, M. (2016). *Microservice architecture: aligning principles, practices, and culture*. "O'Reilly Media, Inc.".

[7] Merkel, Dirk. *Docker: lightweight linux containers for consistent development and deployment.*, Linux journal, 2014.239 (2014): 2.

[8] Masse, Mark. *REST API Design Rulebook: Designing Consistent RESTful Web Service Interfaces*. " O'Reilly Media, Inc.", 2011.

[9] Pezoa, F., Reutter, J. L., Suarez, F., Ugarte, M., & Vrgoč, D. (2016, April). *Foundations of JSON schema.* In Proceedings of the 25th International Conference on World Wide Web (pp. 263-273).

[10] Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., ... & Kudlur, M. (2016). *Tensorflow: A system for large-scale machine learning,* In 12th {USENIX} symposium on operating systems design and implementation ({OSDI} 16) (pp. 265-283).

[11] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Desmaison, A. (2019). *Pytorch: An imperative style, high-performance deep learning library,* In Advances in neural information processing systems (pp. 8026-8037).

[12] Yoo, A. B., Jette, M. A., & Grondona, M. (2003, June). *Slurm: Simple linux utility for resource management,* In Workshop on Job Scheduling Strategies for Parallel Processing (pp. 44-60). Springer, Berlin, Heidelberg.

[13] Kenler, Emilien, and Federico Razzoli. *MariaDB Essentials*. Packt Publishing Ltd, 2015.

[14] Li, Qing, and Yu-Liu Chen. "Entity-relationship diagram." *Modeling and Analysis of Enterprise and Information Systems*. Springer, Berlin, Heidelberg, 2009. 125-139.

[15] Pooley, Rob, and Peter King. *The unified modelling language and performance engineering,* IEE Proceedings-Software 146.1 (1999): 2-10.

[16] D. Li, X. Chen, M. Becchi and Z. Zong, *Evaluating the Energy Efficiency of Deep Convolutional Neural Networks on CPUs and GPUs,* 2016 IEEE International Conferences on Big Data and Cloud Computing (BDCloud), Social Computing and Networking (SocialCom), Sustainable Computing and Communications (SustainCom) (BDCloud-SocialCom-SustainCom), Atlanta, GA, 2016, pp. 477-484, doi: 10.1109/BDCloud-SocialCom-SustainCom.2016.76.

[17] Jahani, A., Lattuada, M., Ciavotta, M., Ardagna, D., Amaldi, E., Zhang, L.: *Optimizing on-demand gpus in the cloud for deep learning applications training.* In: ICCCS 2019. pp. 1–8. IEEE (Oct 2019). https://doi.org/10.1109/CCCS.2019.8888151